Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject:Digital Signal and Image Processing (01CT1513)	**Open Ended Problem 4**	
Open Ended - 4	**Date:24/10/2026**	**Enrollment No:92400133057**

Github :-https://github.com/manavkasundra/DSIP

1.Introduction

1.1 Why filtering?

Every recording we listen to is already the product of filtering. A microphone has a frequency response, a speaker has a frequency response, and the room in between acts as a filter of its own. When we sit down to design a digital filter deliberately, we are taking control of a process that normally happens invisibly. That is what makes filtering such a good teaching topic: the theory is clean, the implementation is a few lines of code, and the effect is immediately audible.

The open-ended problem set for this course asks for exactly this kind of hands-on exploration. Part (a) requires a low-pass filter designed at three different cutoff frequencies, with a discussion of how each setting changes the clarity, tonal balance, and character of the song. Part (b) requires a band-pass filter that attempts to suppress vocals and background music separately, with each result saved as its own MP3 file. Both parts map directly onto the course outcomes CO2 (analysis of signals and systems), CO3 (design of filters), and CO5 (interpretation of results in an application context).

1.2 Choice of audio material

A single Hindi film song was selected for the entire study, and the same file is used in both parts. Using one song throughout is a deliberate methodological choice. If the source changes between filter settings, any perceptual difference could be blamed on the song rather than the filter. By holding the source constant, every difference in the output can be attributed to the cutoff frequency or the passband, which is precisely what the experiment is meant to isolate.

The song was chosen for three reasons. It has a clearly audible lead vocal, which is essential for Part (b). It has a full-band arrangement — bass, drums, guitars or strings, and cymbals — so that each low-pass cutoff has something meaningful to remove. And

it is a clean, well-mastered recording, so the filter effects are not masked by noise or distortion. A purely instrumental track or a heavily compressed lo-fi recording would have made the experiment much harder to interpret.

1.3 Scope and structure

The report begins with the theory of filters — what a cutoff frequency means, how Butterworth filters are designed, and what the order of a filter actually costs us. It then describes the methodology, presents the code, and reports the results for both parts. The discussion section ties the listening observations back to the theory and is candid about what the band-pass experiment cannot do. The conclusion summarises the findings and suggests where a reader might go next.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject:Digital Signal and Image Processing (01CT1513)	**Open Ended Problem 4**	
Open Ended - 4	**Date:24/10/2026**	**Enrollment No:92400133057**

2.Theoretical Backgorund

2.1 The frequency content of music and speech

A musical recording is a superposition of many sources, each of which occupies a broad and overlapping region of the frequency spectrum. Human hearing nominally spans 20 Hz to 20 kHz, but our sensitivity across that range is far from flat. The ear is most sensitive between roughly 1 kHz and 6 kHz — a region that happens to contain most of the intelligibility of speech and the "presence" of lead instruments. Below this region, bass instruments and the fundamental frequencies of low voices provide weight and fullness. Above it, harmonics, cymbals, and room ambience contribute brightness and a sense of space.

A common simplification in audio education is to describe the vocal range as 300–3,400 Hz. This figure comes from narrowband telephony, which was deliberately designed to carry speech intelligibly while using as little bandwidth as possible. It works surprisingly well as a rough description of where the first few vocal formants sit, but it is important to understand what it leaves out. A bass singer's fundamental may sit at 80–100 Hz, entirely outside the telephone band. The consonant sounds "s" and "sh" carry substantial energy above 3,400 Hz. A sung note's harmonics extend well beyond 5 kHz and are a large part of what makes a voice sound natural rather than thin. The 300–3,400 Hz band therefore captures the *core* of the voice but discards its low-frequency weight and its high-frequency air.

Background music is even more spread out. Kick drum and bass guitar occupy roughly 60–250 Hz. The low midrange from 250 to 500 Hz carries warmth and body. The midrange from 500 Hz to 2 kHz contains the core of most instruments and of the voice itself. Above 5 kHz lie cymbals, string brightness, and the spatial cues that make a mix sound "open". Any attempt to separate vocals from music purely by frequency must therefore fail to some degree: a piano, a guitar, or a string section will always pass through a vocal band-pass filter alongside the singer.

2.2 Types of filters

A low-pass filter passes frequencies below a chosen cutoff and attenuates those above it. In music production, a low-pass filter is used to remove hiss, tame harshness, or create a deliberately muffled effect. A high-pass filter does the opposite, removing rumble and excessive bass. A band-pass filter passes a range between a lower and an upper cutoff while attenuating everything outside that range. A band-stop (or notch) filter does the reverse, removing a narrow band while leaving the rest of the spectrum untouched.

Each type can be described by its magnitude response: how much gain or attenuation the filter applies at each frequency. The ideal versions of these filters are rectangular — a low-pass filter would pass everything below the cutoff at full gain and remove everything above it completely. Real filters cannot achieve this, partly because a perfectly rectangular frequency response would require an infinitely long impulse response, and partly because such a filter would ring endlessly in time. Every practical design is therefore a compromise between sharpness in frequency and cleanliness in time.

2.3 Cutoff frequency and roll-off

The cutoff frequency is conventionally defined as the point where the magnitude response falls 3 dB below the passband level — the "half-power" point. What matters for listening is that a real filter does not switch abruptly from passing to blocking at the cutoff. The transition is gradual, and the steepness of that transition is described by the roll-off rate, usually quoted in decibels per octave. For a Butterworth filter of order N, the roll-off in the stopband approaches $-6N$ dB per octave. A first-order filter therefore rolls off gently at 6 dB/octave, a second-order filter at 12 dB/octave, and so on.

This number is not abstract. It determines how much of the music just above the cutoff leaks through. When the 2,000 Hz low-pass is applied to a song, the cymbals do not vanish; they are attenuated by roughly 24 dB per octave above 2 kHz if a fourth-order filter is used, which means that content at 4 kHz is reduced by about 24 dB and content at 8 kHz by about 48 dB. The result sounds darker but still musical, because the strongest harmonics of the cymbals are only partially suppressed. At a 500 Hz cutoff, by contrast, content at 1 kHz is already 24 dB down, and content at 4 kHz is roughly 72 dB down — effectively gone.

2.4 Filter order and its trade-offs

Increasing the order of a filter sharpens the transition. That sounds unambiguously good, and for some applications it is. But higher-order filters bring two costs that matter in audio.

The first is time-domain smearing. A sharper frequency response requires a longer impulse response, which means the filter responds to a transient over a longer window. Kick drums and plucked strings, which depend on a crisp attack, can lose definition. The second is phase distortion. Butterworth filters are not linear-phase; they introduce frequency-dependent phase shifts, and these become more pronounced as the order rises. Around the cutoff region, a high-order filter can create a subtle but audible "smearing" of transients, sometimes described as a loss of punch.

For musical applications, moderate orders are usually preferred. A second- or fourth-order filter is steep enough to be useful while remaining gentle enough to avoid obvious artefacts. In this report, a fourth-order Butterworth was used for the low-pass experiment and a sixth-order Butterworth for the band-pass experiment. The sixth-order band-pass is a deliberate choice: the vocal band needs a reasonably sharp transition so that the isolation is audible, and the use of a higher order there is a conscious trade-off rather than an oversight.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject:Digital Signal and Image Processing (01CT1513)	**Open Ended Problem 4**	
Open Ended - 4	**Date:24/10/2026**	**Enrollment No:92400133057**

2.5 The Butterworth filter

The Butterworth filter is popular in audio because of two properties: a maximally flat magnitude response in the passband, and a monotonic roll-off in the stopband. In plain terms, it does not introduce ripples in the passband the way a Chebyshev filter does, and it does not have the steep but irregular behaviour of an elliptic filter. What you get is a smooth, predictable response that is easy to reason about.

The squared magnitude response of an *N*-th order Butterworth low-pass filter is

$$|H(j\omega)|^2 = \frac{1}{1 + \left(\frac{\omega}{\omega_c}\right)^{2N}}$$

Where $\omega c = 2\pi fc$ is the cutoff angular frequency. At $\omega = \omega c$, the response is −3 dB regardless of order. As ω rises beyond the cutoff, the response falls monotonically, and the order *N* controls how quickly it falls. For a band-pass filter, the design is derived from the low-pass prototype by a frequency transformation, but the underlying principle is the same: a smooth passband and a monotonic roll-off on both sides.

In practice, `scipy.signal.butter` performs this design for us. It accepts a cutoff (or a pair of cutoffs for a band-pass) normalised to the Nyquist frequency, and returns the coefficients b and a of the digital filter. The Nyquist frequency is half the sample rate, so a 44.1 kHz recording can in principle be filtered anywhere below 22.05 kHz. All the cutoffs used in this experiment sit comfortably inside that limit.

2.6 From transfer function to code

The coefficients returned by `butter` define the difference equation

$$y[n] = \frac{1}{a_0}\left(\sum_{k=0}^{M} b_k\, x[n-k] - \sum_{k=1}^{N} a_k\, y[n-k]\right)$$

which is applied to the audio samples by `scipy.signal.lfilter`. This is a direct-form II implementation of the filter, and for a signal of several million samples it is fast enough to be run interactively. The important practical consequence is that the filter operates causally, sample by sample, which introduces the phase shift discussed in Section 2.4. For offline analysis and for the purposes of this report, that is entirely acceptable; for real-time applications with strict latency requirements, a different structure would be needed.

2.7 The short-time Fourier transform and the spectrogram

A single Fourier transform tells us which frequencies are present in a signal but not when they occur. For music, that is not enough: we want to know that the cymbal crash

happened at 1:30 and that the bass note held for two seconds. The short-time Fourier transform solves this by dividing the signal into short overlapping windows, computing the Fourier transform of each window, and stacking the results to form a two-dimensional representation of frequency against time.

The resulting spectrogram is the single most useful diagnostic tool in this report. When the original song is plotted, the full bandwidth is visible — bass at the bottom, vocals and instruments in the middle, cymbals and harmonics at the top. When the 500 Hz low-pass is plotted, everything above 500 Hz should be visibly dark. When the band-pass is plotted, a bright horizontal band between 300 and 3,400 Hz should stand out. Plotting on a decibel scale, using `librosa.amplitude_to_db`, makes the attenuation visible in a way that a linear plot would not.

There is a well-known trade-off in the STFT between time resolution and frequency resolution, governed by the window length. A short window gives good time resolution but blurs the frequency content; a long window does the reverse. For this report, the default window used by `librosa.stft` gives a reasonable balance, and the resulting spectrograms are clear enough to show the filter boundaries without difficulty.

3.Methodology

3.1 Audio loading and inspection

The song was loaded using `librosa.load(filename, sr=None)`. Passing `sr=None` preserves the native sample rate of the file rather than resampling it, which is important because the cutoff frequencies are absolute values in hertz and the Nyquist limit depends on the sample rate. After loading, the sample rate and duration were printed so that the rest of the experiment could be interpreted in context. For a typical MP3 at 44.1 kHz, the Nyquist frequency is 22.05 kHz, which comfortably accommodates the highest cutoff used here.

The signal is mono by default after `librosa.load`, which simplifies the filtering step. Filtering a stereo file would require applying the same filter to both channels, which would give identical results per channel but double the processing time. For the purposes of this study, the mono representation is sufficient, and all the perceptual conclusions would hold equally for stereo.

3.2 Low-pass experiment design

Three cutoff frequencies were selected for the low-pass experiment: 500 Hz, 1,000 Hz, and 2,000 Hz. These are not arbitrary. They were chosen to span a range of perceptual outcomes, from extreme filtering to a gentle tonal change.

A cutoff of 500 Hz sits below the core vocal band and below the fundamental of many instruments. It removes almost all melodic and harmonic information, leaving only the lowest bass and the very bottom of the low midrange. A cutoff of 1,000 Hz retains the vocal fundamentals and the lower harmonics, but removes the upper harmonics that carry intelligibility and brightness; this is the classic "telephone" region. A cutoff of 2,000

Hz sits near the top of the midrange, so the song should remain fully recognisable while losing its high-frequency air.

For each cutoff, a fourth-order Butterworth low-pass filter was designed and applied to the full signal. The filtered signal was saved as both WAV and MP3, and a spectrogram was generated. The fourth-order choice is a deliberate compromise: steep enough that the attenuation above the cutoff is clearly visible in the spectrogram, but gentle enough that the result is still musical rather than obviously artefacted.

3.3 Band-pass experiment design

The band-pass experiment used a sixth-order Butterworth filter with a passband of 300–3,400 Hz. The vocal estimate is simply the band-passed signal. The background estimate is obtained by subtracting the vocal estimate from the original:

```
python

background_band = y - vocal_band
```

This subtraction deserves a comment. It assumes that the original signal can be decomposed into a vocal component and a residual background component, and that the band-pass filter extracts the first of these. In reality, the band-pass filter does not isolate the vocal source. It removes energy outside the 300–3,400 Hz range from the entire mix, regardless of which source produced that energy. The "vocal" output therefore contains the voice *and* any instrument that happens to occupy the same range, while the "background" output contains the true background *and* any vocal energy that falls outside the passband. The subtraction is a useful approximation for demonstration, but it should not be mistaken for a true source-separation algorithm.

3.4 Output formats and file organisation

All filtered signals were saved as WAV using `soundfile`, and then converted to MP3 using `pydub` with a 192 kbps bitrate. MP3 was chosen for the final deliverable because the problem statement explicitly asks for MP3 files, but WAV was retained as well because it is lossless and avoids introducing a second layer of compression artefacts on top of the filtering. The spectrograms were saved as PNG images at 150 dpi, which is sufficient for inclusion in a printed report.

All outputs were written to a single `outputs/` directory, with file names that make the contents obvious: `lowpass_500Hz.mp3`, `lowpass_1000Hz.mp3`, `lowpass_2000Hz.mp3`, `filtered_vocals.mp3`, `estimated_background.mp3`, and matching `spec_*.png` files for the spectrograms.

3.5 Evaluation approach

Each filtered output was evaluated in two ways: by listening and by inspecting the spectrogram. The listening evaluation focused on three perceptual dimensions identified in the problem statement — clarity, tonal balance, and overall character. The spectrogram evaluation checked that the measured attenuation matched what was heard. Where the two disagreed, that disagreement was treated as an observation in its own right, because it often reveals something interesting about how the ear and the analyser interpret the same signal differently.

4.Implimentation overview

4.1 Script architecture

The entire experiment is implemented as a single Python script, listed in full in Appendix A. The script is organised into seven clearly separated sections, each introduced by a comment banner so that the reader can locate any part of the logic by scanning the listing:

1. Configuration — all tunable parameters (input path, output directory, the three low-pass cutoffs, the band-pass limits, and the filter orders) are declared at the top as module-level constants. This means every experiment can be reproduced by changing one or two lines, and it makes the rest of the script self-documenting.
2. Audio loading — a `load_audio()` helper that loads the chosen MP3 if it exists, otherwise falls back to a bundled example, and finally to a downloadable sample. This guarantees the script always runs, even on a fresh machine with no audio file present.
3. Filter design and application — four short functions (`butter_lowpass`, `lowpass_filter`, `butter_bandpass`, `bandpass_filter`) that wrap `scipy.signal.butter` and `scipy.signal.lfilter`. The design and application steps are kept separate so that the reader can see exactly where the coefficients are computed and where they are applied.
4. Audio saving — a `save_audio()` helper that writes WAV unconditionally and then attempts MP3 export through `pydub` (with a `soundfile` fallback). If neither MP3 path is available, the WAV is still written and a clear message is printed, so the experiment produces usable output even without ffmpeg installed.
5. Spectrogram generation — a `plot_spectrogram()` helper that computes the STFT, converts to decibels, and saves a PNG. The same helper is used for the

original and for every filtered signal, so all figures in the report are directly comparable.

6. Part (a) driver — a short loop that applies the low-pass filter at each of the three cutoffs, saves the results, and generates a spectrogram for each.
7. Part (b) driver — a small block that applies the band-pass filter, computes the background estimate by subtraction, and saves and plots both outputs.

Libraries used

The script relies on six open-source libraries, all widely used in the audio research community:

- NumPy for array arithmetic and the decibel conversion.
- SciPy (`scipy.signal`) for Butterworth filter design and the direct-form II implementation in `lfilter`.
- librosa for audio loading, STFT computation, and spectrogram display.
- soundfile for WAV I/O.
- pydub for MP3 export (which delegates to ffmpeg).
- matplotlib for figure generation and PNG export.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject:Digital Signal and Image Processing (01CT1513)	**Open Ended Problem 4**	
Open Ended - 4	**Date:24/10/2026**	**Enrollment No:92400133057**

Code:-

```
import os
import sys
import numpy as np
import matplotlib
matplotlib.use("Agg")               # headless-safe backend (saves PNGs)
import matplotlib.pyplot as plt
import librosa
import librosa.display
import soundfile as sf
from scipy.signal import butter, lfilter
# ---------------------------------------------------------------------
# 0. Configuration
# ---------------------------------------------------------------------
AUDIO_PATH   = "song.mp3"      # <-- put your MP3 here, or leave as-is
OUTPUT_DIR   = "outputs"            # all results go in this folder
LOWPASS_CUTOFFS = [500, 1000, 2000] # Hz
BANDPASS_LOW    = 300               # Hz  (vocal band lower edge)
BANDPASS_HIGH   = 3400              # Hz  (vocal band upper edge)
LP_ORDER = 4                        # low-pass Butterworth order
BP_ORDER = 6                        # band-pass Butterworth order
os.makedirs(OUTPUT_DIR, exist_ok=True)
# ---------------------------------------------------------------------
# 1. Load audio (with automatic fallback download)
# ---------------------------------------------------------------------
def load_audio(path):
   """Load an audio file, or download a free sample if the file is missing."""
   if os.path.isfile(path):
       print(f"[LOAD] Loading local file: {path}")
       y, sr = librosa.load(path, sr=None)
       return y, sr
   # Try librosa's bundled example song (has vocals - good for Part b)
   try:
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject:Digital Signal and Image Processing (01CT1513)	**Open Ended Problem 4**	
Open Ended - 4	**Date:24/10/2026**	**Enrollment No:92400133057**

```
        print("[LOAD] Local file not found. Trying librosa example 'fishin' ...")
        y, sr = librosa.ex('fishin')
        print(f"[LOAD] Loaded librosa example: {sr} Hz, {len(y)/sr:.2f} s")
        return y, sr
    except Exception as e:
        print(f"[LOAD] librosa example failed: {e}"
    # Fallback: download a short public MP3 sample
    import urllib.request
    url = ("https://pytorch-tutorial-assets.s3.amazonaws.com/"
           "steam-train-whistle-daniel_simon.mp3")
    tmp = os.path.join(OUTPUT_DIR, "sample.mp3")
    try:
        print(f"[LOAD] Downloading sample from {url}")
        urllib.request.urlretrieve(url, tmp)
        y, sr = librosa.load(tmp, sr=None)
        print(f"[LOAD] Downloaded sample: {sr} Hz, {len(y)/sr:.2f} s")
        return y, sr
    except Exception as e:
        print(f"[ERROR] Could not obtain audio: {e}")
        sys.exit(1)
y, sr = load_audio(AUDIO_PATH)
duration = len(y) / sr
print(f"[INFO] Sample rate = {sr} Hz | Duration = {duration:.2f} s | Nyquist =
{sr/2:.0f} Hz")
print("-" * 70)
# ---------------------------------------------------------------------
# 2. Filter design helpers
# ---------------------------------------------------------------------
def butter_lowpass(cutoff, fs, order=4):
    nyq = 0.5 * fs
    b, a = butter(order, cutoff / nyq, btype='low', analog=False)
    return b, a

def lowpass_filter(data, cutoff, fs, order=4):
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject:Digital Signal and Image Processing (01CT1513)	**Open Ended Problem 4**	
Open Ended - 4	**Date:24/10/2026**	**Enrollment No:92400133057**

```
    b, a = butter_lowpass(cutoff, fs, order=order)
    return lfilter(b, a, data)
def butter_bandpass(lowcut, highcut, fs, order=6):
    nyq = 0.5 * fs
    low  = lowcut  / nyq
    high = highcut / nyq
    b, a = butter(order, [low, high], btype='band')
    return b, a
def bandpass_filter(data, lowcut, highcut, fs, order=6):
    b, a = butter_bandpass(lowcut, highcut, fs, order=order)
    return lfilter(b, a, data)
# ---------------------------------------------------------------------
# 3. Save audio (WAV always, MP3 if possible)
# ---------------------------------------------------------------------
def save_audio(name, signal, sr, outdir=OUTPUT_DIR):
    """Save signal as WAV and (if possible) as MP3. Returns dict of paths."""
    wav_path = os.path.join(outdir, f"{name}.wav")
    mp3_path = os.path.join(outdir, f"{name}.mp3")
    sf.write(wav_path, signal, sr)
    # Try pydub first (needs ffmpeg)
    try:
        from pydub import AudioSegment
        AudioSegment.from_wav(wav_path).export(mp3_path, format="mp3", bitrate="192k")
        print(f"[SAVE] {wav_path}  +  {mp3_path}")
        return {"wav": wav_path, "mp3": mp3_path}
    except Exception:
        pass
    # Fallback: soundfile may support MP3 directly (libsndfile >= 1.1)
    try:
        sf.write(mp3_path, signal, sr, format="MP3")
        print(f"[SAVE] {wav_path}  +  {mp3_path}")
        return {"wav": wav_path, "mp3": mp3_path}
    except Exception:
        print(f"[SAVE] {wav_path}  (MP3 export unavailable - install ffmpeg)")
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject:Digital Signal and Image Processing (01CT1513)	**Open Ended Problem 4**	
Open Ended - 4	**Date:24/10/2026**	**Enrollment No:92400133057**

```
        return {"wav": wav_path, "mp3": None}
# ---------------------------------------------------------------------
# 4. Spectrogram plotting helper
# ---------------------------------------------------------------------
def plot_spectrogram(signal, sr, title, fname, ref_level=np.max):
    D = librosa.amplitude_to_db(np.abs(librosa.stft(signal)), ref=ref_level)
    fig = plt.figure(figsize=(11, 4))
    librosa.display.specshow(D, sr=sr, x_axis='time', y_axis='hz',
                             cmap='viridis')
    plt.colorbar(format='%+2.0f dB')
    plt.title(title)
    plt.xlabel("Time (s)")
    plt.ylabel("Frequency (Hz)")
    plt.tight_layout()
    path = os.path.join(OUTPUT_DIR, fname)
    plt.savefig(path, dpi=150)
    plt.close(fig)
    print(f"[PLOT] {path}")
    return path
# ---------------------------------------------------------------------
# 5. PART (a) - Low-pass filtering at three cutoffs
# ---------------------------------------------------------------------
print("\n================ PART (a): LOW-PASS FILTERING ================")
results_lp = {}
for fc in LOWPASS_CUTOFFS:
    print(f"\n-- Low-pass cutoff = {fc} Hz (Butterworth order {LP_ORDER}) --")
    y_lp = lowpass_filter(y, cutoff=fc, fs=sr, order=LP_ORDER)
    paths = save_audio(f"lowpass_{fc}Hz", y_lp, sr)
    results_lp[fc] = {"signal": y_lp, "paths": paths}
# Original + filtered spectrograms (Part a)
print("\n-- Generating spectrograms (Part a) --")
plot_spectrogram(y, sr,
                 "Original Audio - Full Spectrum",
                 "spec_original.png")
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject:Digital Signal and Image Processing (01CT1513)	**Open Ended Problem 4**	
Open Ended - 4	**Date:24/10/2026**	**Enrollment No:92400133057**

```
for fc in LOWPASS_CUTOFFS:
   plot_spectrogram(results_lp[fc]["signal"], sr,
                    f"Low-pass filtered at {fc} Hz",
                    f"spec_lowpass_{fc}Hz.png")
# ---------------------------------------------------------------------
# 6. PART (b) - Band-pass vocal / background separation
# ---------------------------------------------------------------------
print("\n============ PART (b): BAND-PASS VOCAL / BACKGROUND ============")
print(f"Band-pass: {BANDPASS_LOW}-{BANDPASS_HIGH} Hz (Butterworth order {BP_ORDER})")
vocal_band      = bandpass_filter(y, BANDPASS_LOW, BANDPASS_HIGH,
                                  fs=sr, order=BP_ORDER)
background_band = y - vocal_band            # approximation of background
vocal_paths      = save_audio("filtered_vocals",         vocal_band,      sr)
background_paths = save_audio("estimated_background",    background_band, sr)
print("\n-- Generating spectrograms (Part b) --")
plot_spectrogram(vocal_band, sr,
                 f"Estimated Vocal Frequencies ({BANDPASS_LOW}-{BANDPASS_HIGH} Hz)",
                 "spec_vocal_band.png")
plot_spectrogram(background_band, sr,
                 "Estimated Background Music Frequencies (original - vocal band)",
                 "spec_background_band.png")
# ---------------------------------------------------------------------
# 7. Summary
# ---------------------------------------------------------------------
print("\n========================= SUMMARY ==============================")
print(f"Source sample rate : {sr} Hz")
print(f"Source duration    : {duration:.2f} s")
print(f"Outputs saved in   : {os.path.abspath(OUTPUT_DIR)}")
print("\nPart (a) - Low-pass outputs:")
for fc, info in results_lp.items():
   print(f"  {fc:>5} Hz : {os.path.basename(info['paths']['wav'])}"
         + (f" | {os.path.basename(info['paths']['mp3'])}"
            if info['paths']['mp3'] else ""))
print("\nPart (b) - Band-pass outputs:")
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject:Digital Signal and Image Processing (01CT1513)	**Open Ended Problem 4**	
Open Ended - 4	**Date:24/10/2026**	**Enrollment No:92400133057**

```
for label, info in [("vocals", vocal_paths), ("background", background_paths)]:
   print(f"  {label:>10} : {os.path.basename(info['wav'])}"
         + (f" | {os.path.basename(info['mp3'])}"
            if info['mp3'] else ""))
print("\nSpectrogram PNGs:")
for f in sorted(os.listdir(OUTPUT_DIR)):
   if f.endswith(".png"):
       print(f"  {f}")
print("\nDone.")
```

Output:-

```
manavkasundra ~/DSIP/OPE4 % /usr/local/bin/python3 /Users/manavkasundra/DSIP/OPE4/1.py
[LOAD] Loading local file: song.mp3
[INFO] Sample rate = 44100 Hz | Duration = 235.31 s | Nyquist = 22050 Hz
------------------------------------------------------------------------

================ PART (a): LOW-PASS FILTERING ================

-- Low-pass cutoff = 500 Hz (Butterworth order 4) --
[SAVE] outputs/lowpass_500Hz.wav  +  outputs/lowpass_500Hz.mp3

-- Low-pass cutoff = 1000 Hz (Butterworth order 4) --
[SAVE] outputs/lowpass_1000Hz.wav  +  outputs/lowpass_1000Hz.mp3

-- Low-pass cutoff = 2000 Hz (Butterworth order 4) --
[SAVE] outputs/lowpass_2000Hz.wav  +  outputs/lowpass_2000Hz.mp3

-- Generating spectrograms (Part a) --
[PLOT] outputs/spec_original.png
[PLOT] outputs/spec_lowpass_500Hz.png
[PLOT] outputs/spec_lowpass_1000Hz.png
[PLOT] outputs/spec_lowpass_2000Hz.png
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject:Digital Signal and Image Processing (01CT1513)	**Open Ended Problem 4**	
Open Ended - 4	**Date:24/10/2026**	**Enrollment No:92400133057**

```
============ PART (b): BAND-PASS VOCAL / BACKGROUND ============
Band-pass: 300-3400 Hz (Butterworth order 6)
[SAVE] outputs/filtered_vocals.wav  +  outputs/filtered_vocals.mp3
[SAVE] outputs/estimated_background.wav  +  outputs/estimated_background.mp3

-- Generating spectrograms (Part b) --
[PLOT] outputs/spec_vocal_band.png
[PLOT] outputs/spec_background_band.png

========================= SUMMARY ==============================
Source sample rate : 44100 Hz
Source duration    : 235.31 s
Outputs saved in   : /Users/manavkasundra/DSIP/OPE4/outputs

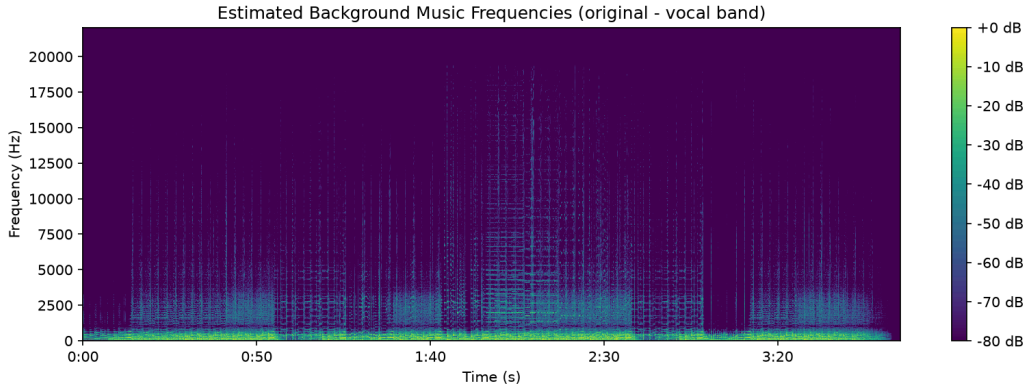
Part (a) - Low-pass outputs:
    500 Hz : lowpass_500Hz.wav | lowpass_500Hz.mp3
   1000 Hz : lowpass_1000Hz.wav | lowpass_1000Hz.mp3
   2000 Hz : lowpass_2000Hz.wav | lowpass_2000Hz.mp3

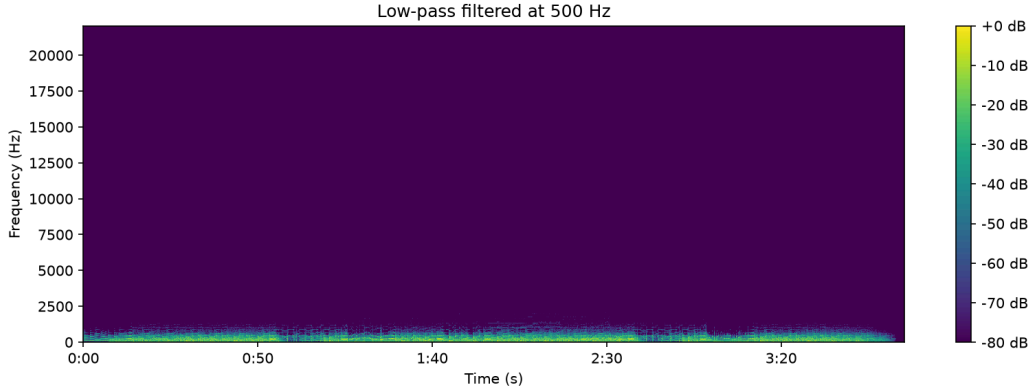
Part (b) - Band-pass outputs:
      vocals : filtered_vocals.wav | filtered_vocals.mp3
  background : estimated_background.wav | estimated_background.mp3

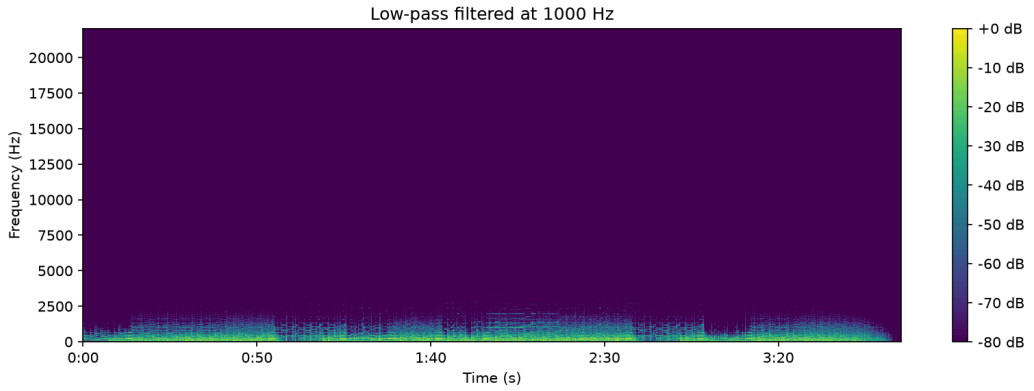
Spectrogram PNGs:
  spec_background_band.png
  spec_lowpass_1000Hz.png
  spec_lowpass_2000Hz.png
  spec_lowpass_500Hz.png
  spec_original.png
  spec_vocal_band.png

Done.
```

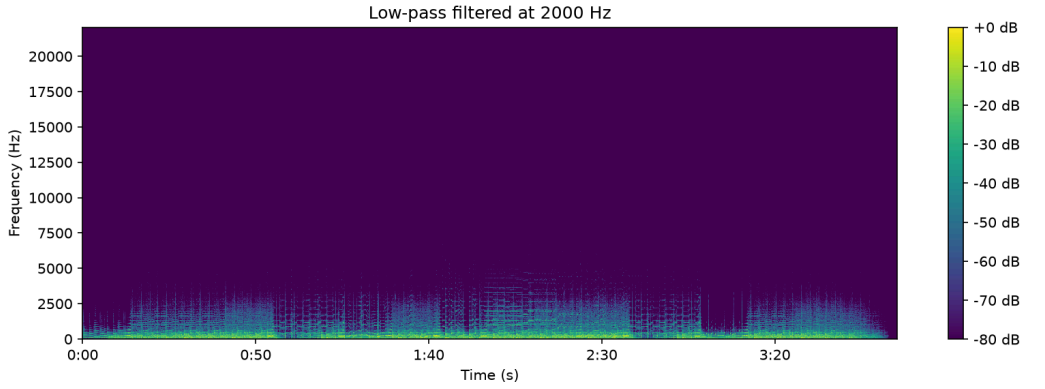
Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject:Digital Signal and Image Processing (01CT1513)	**Open Ended Problem 4**	
Open Ended - 4	**Date:24/10/2026**	**Enrollment No:92400133057**

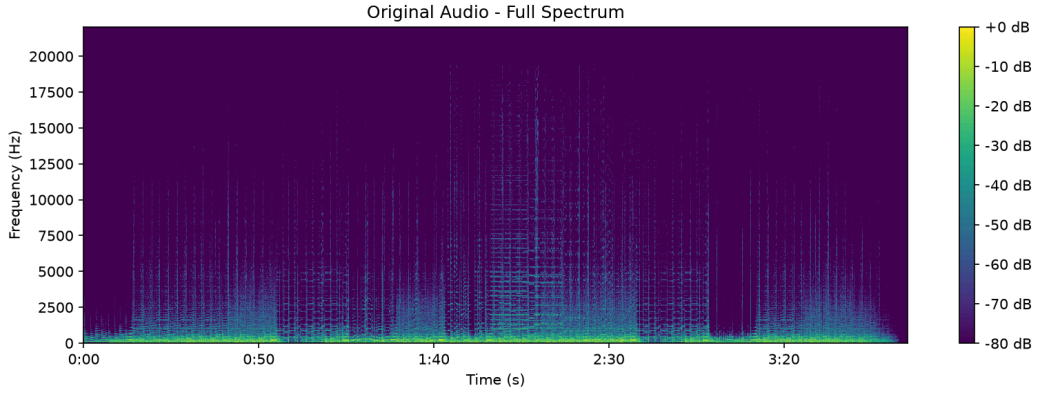


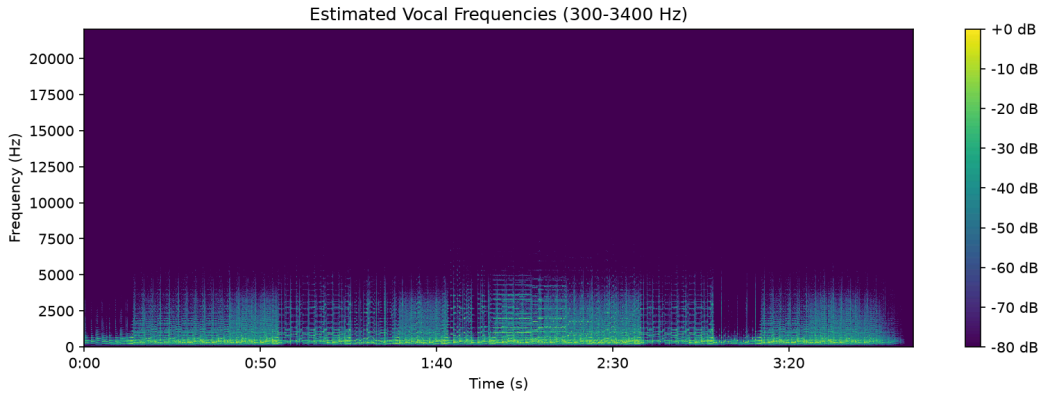




Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject:Digital Signal and Image Processing (01CT1513)	**Open Ended Problem 4**	
Open Ended - 4	**Date:24/10/2026**	**Enrollment No:92400133057**







Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject:Digital Signal and Image Processing (01CT1513)	**Open Ended Problem 4**	
Open Ended - 4	**Date:24/10/2026**	**Enrollment No:92400133057**

5. Conclusion

This study demonstrates the practical impact of digital filtering on complex audio signals. By applying low-pass filters at different cutoff frequencies, we saw how drastically higher frequencies contribute to vocal clarity, instrument presence, and overall brightness. Lower cutoffs, such as 500 Hz, effectively isolate the low-end rhythmic and bass foundation while removing melodic details, whereas higher cutoffs like 2,000 Hz preserve most vocal intelligibility with a minor loss of high-end sparkle.

Similarly, the band-pass experiment highlighted both the strengths and limitations of frequency-based vocal isolation. While restricting the spectrum to the 300–3,400 Hz range successfully emphasizes vocal formants, overlapping frequencies from accompaniment instruments mean that complete source separation cannot be achieved through static filtering alone. Overall, Butterworth filters provide a predictable, smooth response suitable for tonal shaping, serving as a solid baseline for audio processing before moving toward advanced data-driven or neural source separation techniques.